[5, 1, 2, 3, 4]

Subarrays

[5] ✓

[1, 2] ✓

[1, 2, 3, 4] ✓

[2, 3, 4] ✓

[2, 4] ✗

[] ✓

[2, 1] ✗

[5, 1] ✓

Subsequence

[1] ✓

[1, 2] ✓

[2, 4] ✓

[5, 2, 4] ✓

[5, 4, 2] ✗

[5, -6, 3, 2, 7, -5, 8]

o/p = 15

i: 0 ⟶ n-1
sum = 0
j: i ⟶ n-1
sum += arr[j]
if sum > maxSum
maxSum = sum

TC = O(n²)
AS = O(1)

[5, -6, 3, 2, 7, -5, 8]

5 12 7 15

arr: [5, -6, 3, 2, 7, -5, 8]

mth: [5 -1 3 5 12 7 15]

TC = O(n)
AS = O(n)

[6, -6, 3, 2]

6 0 3 5

mth[i] = max(mth[i-1] + arr[i], arr[i])

```
int getMaxSumSubarray(vector<int> arr) {
    int n = arr.size();
    int maxSum = arr[0];
    int result = arr[0];

    for (int i = 1; i < n; i++) {
        maxSum = max(maxSum + arr[i], arr[i]);
        result = max(result, maxSum);
    }

    return result;
}
```

mth = 5 -1 3 5 12 7 15

ans = 5 12 15

[5, -6, 3, 2, 7, -5, 8]
0 1 2 3 4 5 6
5 -1 3 5 12 7 15 n

[-8, 8, -9, 9, -10, 11, -12]

0 1 2 3 4 5 6
[-8, -8, 9, -9, 10, -11, 12] n = 7

o/p = 22

i = (i+1)% n

[-8, 10, 6, -5, -6] o/p = 16

---

case-1 Apply normal Kadane's algo
Suppose that gives maxSum = sum1

case-2 Use Kadane's to get the minSum of any subarray
sum2 = arrSum - minSum

return max(sum1, sum2)

Prefix / Suffix

arr: [1, 5, 3, -2, 6, 8]
prefix: [1, 6, 9, 7, 13, 21]
suffix: [21, 20, 15, 12, 14, 8]

[1, 15] → 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 - - - 15
[6, 8] → 3, 4, 5, 6, 7, 8
[3, 5] → 3, 4, 5
[1, 4] → 1, 2, 3, 4

o/p = 3

[1, 3] → 1, 2, 3
[2, 3] → 2, 3
[6, 8] → 6, 7, 8

o/p = 2

TC: O(n + maxx)

1 → 1
2 → ~~1~~ 2
3 → ~~1~~ 2
6 - 1
7 - 1
8 - 1

[1, 3] [2, 3]

freq:

0	1	2	3	4	5	6	7	8	9
	+1	+1		-2		+1			-1
0	1	2	2	0	0	1	1	1	0

TC: O(n + maxx)

---

0 1 2 3
nums: [1, 2, 3, 4]
ans: [24, 12, 8, 6]

nums [-1, 1, 0, -3, 3]
ans: [0, 0, 9, 0, 0]

[0, -1, 0, -3, 3]
[0, 0, 0, 0, 0]

BF

i: 0 ⟶ n-1
j: 0 ⟶ i-1
. . . . .
j: i+1 ⟶ n-1
. . . .

TC = O(n²)
AS = O(1)

Pref / Suffix Prod

TC = O(n)
AS = O(n)

0 n-2 n-1

prefixProd[i] = arr[0] × arr[1] - - - × arr[i-1]
suffixProd[i] = arr[i+1] × arr[i+2] - - - × arr[n-1]
ans[i] = prefixProd[i] × suffixProd[i]

[1, 3, 4, 10], target = 8 ✗

[1, 3, 4, 4, 10], target = 8 ✓

i: 0 ⟶ n-1
j: i+1 ⟶ n-1
if a[i] + a[j] == target
return true.

TC = O(n²)
AS = O(1)

i = n-2
j = n-1
k = n-1

0, 1, 2, 3, ... (n-3), (n-2), (n-1)